

08-docker-realtime.md

Docker Networking — ELI5 + Real World Map

The Big Picture — Apartment Building Analogy

Imagine Docker containers are **people living in apartments**. Networks are the **roads/lanes** connecting them.

Without network: Everyone **locked in** their own room. No talking. ❌

With network: Everyone **on** same street. Can talk freely. ✅

Why Docker Network Exists — The Problem Story

Day 1:

App Container → talks to → DB Container (IP: 172.17.0.2) ✅

Day 2: DB container crashes → new one created

New DB Container IP = 172.17.0.4 ← CHANGED!

App still tries 172.17.0.2 → ❌ DEAD CONNECTION

Solution = Docker Network

App talks to DB by NAME **not** IP

Name never changes even if IP changes ✅

Real world: Instead of saving someone's mobile number, you save their **name in contacts**. Even if they change number, you just update once.

4 Types of Networks — Road Types

BRIDGE	= Private colony road (default)
HOST	= National highway (share EC2's IP)
NONE	= No road at all (total isolation)
OVERLAY	= Inter-city highway (across servers)

Network	Real World	Use Case
Bridge	Private gated colony	Containers on same server talking
Host	Container = EC2 itself	No port mapping, max speed

Network	Real World	Use Case
None	Bunker with no doors	Total isolation, security
Overlay	Fiber cable between cities	Swarm, multi-server containers

Bridge Network — Default Colony

EC2 Server (192.168.1.100)

└─ Docker Bridge (172.17.0.0)

└─ cont1 → 172.17.0.2

└─ cont2 → 172.17.0.3

└─ cont3 → 172.17.0.4

cont1 ↔ cont2  (same colony, same road)

bash

By default every container goes to bridge

```
docker run -itd --name cont1 ubuntu
```

```
docker run -itd --name cont2 ubuntu
```

Check IP

```
docker inspect cont1    # look for "IPAddress"
```

Host Network — Container = EC2 Twin

BRIDGE:

EC2 IP: 192.168.1.100

Container IP: 172.17.0.2

HOST:

EC2 IP: 192.168.1.100

Container IP: 192.168.1.100 (SAME!)

Need port mapping: -p 80:80 No port mapping needed!

bash

```
docker run -d --network host --name myapp nginx
```

nginx now runs ON ec2's IP directly

access via: http://192.168.1.100:80 (no -p needed)

None Network — The Bunker

bash

```
docker run -d --network none --name secret-cont ubuntu
```

```
# This container:
```

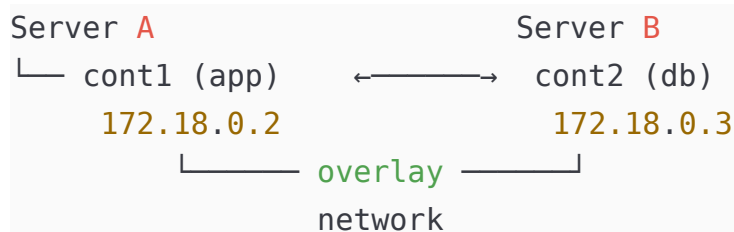
```
# ❌ Can't reach internet
```

```
# ❌ Can't reach other containers
```

```
# ✅ Totally isolated
```

Real world: A server room with **no ethernet cable, no wifi, no nothing**.

🌐 Overlay Network — Inter-City Highway



Only works with Docker Swarm!

🔧 Hands-On: Ping Test Story

Setup:

```
bash
```

```
# Create containers (go to bridge by default)
```

```
docker run -itd --name cont1 ubuntu
```

```
docker run -itd --name cont2 ubuntu
```

```
# cont1 = 172.17.0.2
```

```
# cont2 = 172.17.0.3
```

Test communication:

```
bash
```

```
# Enter cont1
```

```
docker attach cont1
```

```
# Install ping
```

```
apt update && apt install iputils-ping -y
```

```
# Ping cont2
```

```
ping 172.17.0.3 #  works! same bridge network
```

The Two-Network Story — The Key Concept

The Problem:

```
bridge network:  cont1(172.17.0.2)  cont2(172.17.0.3)
flm network:     flm-cont1(172.18.0.2)
```

```
flm-cont1 → ping cont2 →  FAIL
```

Why? Different roads! Like trying to drive from a private colony road onto a highway directly.

The Fix — Connect the Roads:

```
bash
```

```
# Add flm network to cont2 (give it a 2nd road)
```

```
docker network connect flm cont2
```

```
# Now cont2 has 2 networks:
```

```
# eth0 = bridge (172.17.0.3)
```

```
# eth1 = flm      (172.18.0.3) ← NEW
```

```
# Now flm-cont1 can reach cont2 via flm road 
```

BEFORE connect:

```
flm-cont1 [flm] ↔ cont2 [bridge only] 
```

AFTER connect:

```
flm-cont1 [flm] ↔ cont2 [bridge + flm] 
same road now!
```

Real world: cont2 got a **second door** opening onto the flm street. Now flm-cont1 can knock on that door. 

The Full Network State Map

BRIDGE NETWORK

cont1	cont2
.0.2	.0.3

(after connect)

FLM NETWORK

flm-cont1
cont2 (added)

cont2 lives on BOTH roads = talks to everyone ✓

IP Math — Why 65,531 containers per network

$2^{32} = \sim 4$ billion total IPs

But one /16 subnet = $2^{16} = 65,536$

Minus reserved IPs = $\sim 65,531$ usable

172.14.0.1 → first container

172.14.0.2 → second

...

172.14.255.255 → last

📋 All Network Commands Cheatsheet

bash

CREATE

`docker network create flm` # make new network

LIST

`docker network ls` # see all networks

INSPECT

`docker network inspect flm` # full details + IPs

```
# CONNECT (add container to network)
```

```
docker network connect flm cont2      # give cont2 a 2nd road
```

```
# DISCONNECT (remove from network)
```

```
docker network disconnect flm cont2   # remove that road
```

```
# PRUNE (delete unused networks)
```

```
docker network prune                  # cleanup empty networks
```

```
# REMOVE
```

```
docker network rm flm                # delete network
```

Jenkins Pipeline with Docker Registry

groovy

```
pipeline {
    agent any
    stages {

        stage('Code') {           // 🍰 get source code
            steps {
                git 'https://github.com/repo.git'
            }
        }

        stage('Build') {          // 👁 cook the image
            steps {
                sh 'docker build -t image1 .'
            }
        }

        stage('Docker Registry') { // 🚀 send to fridge
            steps {
```


Concept	Remember As
Bridge	Private colony — containers talk within same server
Host	Container becomes EC2 twin — shares real IP
None	Bunker — zero connection
Overlay	Inter-city road — needs Swarm
<code>network connect</code>	Give container a 2nd door
<code>network disconnect</code>	Remove that door
Same network = talk	Same street = neighbours can meet
Different network = no talk	Different city = can't walk over

One line to rule them all: "Same network = same street = they can talk. Different network = different city = they can't." 🏙️